Generalized Ensemble Optimization

Matthew Overing

MSc Computational Finance

School of Computer Science & Electronic Engineering

University of Essex

August 27, 2019

Abstract

This paper suggests a generalized framework for finding the optimal group of combined models for any given set of model or dataset. To begin using the Generalizable Ensemble Optimizer (GEO) three variables must be provided: a dataset split into training, cross-validation and test sets, a collection of models to be optimized and an objective function to evaluate the models' performances. The first step of the algorithm finds the optimal hyperparameter settings for each individual model provided using Bayesian optimization. The second step combines each individual models' predictions into a single and more predictive model. The final step searches for additional models which improve the performance of the blended set of algorithms found in step 2. The machine learning algorithms used in this implementation of the GEO are logistic regression, support vector machine and XGBoost trees along with a Numerai Competition dataset. The results of the algorithm were tested against a benchmark which took the average accuracy of the same three models with the same dataset, but with randomly chosen hyperparameters. The Generalized Ensemble Optimizer was found to outperform its benchmark.

Contents

Abstract i
Motivation & Literature 1
Brief History of Machine Learning Ensembles 1
Ensembles and Portfolio Theory 2
Math of Ensembles 4
Hyperparameter Selection Methods 5
Population Based Training of Neural Networks 8
Evolutionary Ensembles: Combining Learning Agents using Genetic Algorithms 9
Method 11
Descriptions of Algorithms Used 12
XGBoost Decision Trees 12
Support Vector Machine Classifier (SVM Classifer) 15
Stochastic Gradient Descent Classifier (SGD Classifer) 17
Bayesian Optimization & Expected Improvement 19
Genetic Algorithms 20
Evaluation 24
Numerai Dataset 24
Numerai Competition 25
Equal Intensity Random Search 26
Results 28
Discussion 33
References 36

Motivation & Literature

The objective of this project is to build an algorithm which solves for the optimal ensemble of models for any given set of models, objective function and data. This project is important because the model selection process is pertinent to all machine learning and quantitative finance problems. Automation of model creation allows practitioners to find an optimal set of models far faster than using existing methods. Hyperparameters are values which change the way machine learning models are fit. The simplest example of a hyperparameter is the learning rate in gradient descent: how far down the gradient to descend on each iteration (Zulkifli, 2018). The algorithm developed in this project addresses two problems simultaneously: finding the optimal model hyperparameters and finding the optimal set of models.

Brief History of Machine Learning Ensembles

Ensemble learning was the catalyst for most breakthroughs in machine learning over the past quarter century. Around the turn of the millennia, algorithms such as Random Forests and Gradient Boosted Machine Decision Trees (GBM Trees) revolutionized the machine learning field (Craib, 2016). Both algorithms were pioneers of ensemble machine learning models: using several weak learners and combining them into a single predictor which could make stronger predictions than any individual model within. The ensemble methodologies for decision trees were computationally efficient and their results were easily comprehensible. Decision tree ensembles usurped neural networks as the dominant machine learning algorithm for over a decade. University of Toronto professor Geoffrey Hinton and his team of researches created an effective neural network ensemble method: dropout (Hinton et al., 2013). Dropout involves training multiple neural networks which have portion of the connections between nodes removed between layers. The average of all predictions is then ensembled into a model which generalizes better than any of the individual neural networks. Dropout ensembles made neural networks

become the dominant machine learning algorithm once again because dropout allowed deep learning to be performed without massively overfitting the training set (Craib, 2016). Hinton and his team surpassed the ensemble trees' performance and achieved best in class performance for vision, natural language processing and a wide range of other tasks. Today, ensemble methods are still achieving state of the art performance. Deep neural network ensembles and regularized boosted tree ensembles are still at the top of the machine learning food chain.

Ensembles and Portfolio Theory

The benefits of using ensembles is pervasive across developing machine learning models and trading strategies. Trading involves a high degree of uncertainty and quantitative traders seek to find edges that have positive expected returns. Finding the best ensemble of uncorrelated bets is imperative for quantitative trading. According to the Kelly Criterion, there are two ways for any speculator to increase expected wealth: either increase the probability a bet pays off or the profitability of the bet paying off (Craib, 2016). Introducing multiple uncorrelated bets does both. If each trading strategy is thought of as a biased coin, investing all capital into one model is like betting everything on one-coin flip. The expected probability of profiting increases toward infinity as the number of independent bets on the biased coin increases. According to Kelly Criterion, the more likely a bet is to pay off, the higher the percentage of wealth should be invested. The more uncorrelated bets, the lower the risk and subsequently, the higher the Kelly optimal exposure to risk should be taken on. Thus, more uncorrelated bets increase both the likelihood of profitability and profitability per trade. Creating an ensemble of uncorrelated machine learning or financial models is a corollary to the biased coin: increasing the number of uncorrelated models vastly increases expected performance.

Ray Dalio, the founder of the largest hedge fund in the world, Bridgewater Capital says that, “Making a handful of good uncorrelated bets that are balanced and leveraged well is the surest way of having a lot of upside without being exposed to unacceptable downside” (Dalio, 2017). Ray labels this principal as “The Holy Grail of Investing.” Figure 1 displays a graph depicting the relationship between number of assets in a portfolio, annualized standard deviation and the correlation between assets. It shows that increasing the number of assets asymptotically decreases portfolio standard deviation and the lower the intercorrelation of assets, the lower the standard deviation asymptotes. Portfolios with lower intercorrelation of assets first order stochastically dominate portfolios with higher intercorrelation. This means that investors for any level of risk utility should prefer portfolios with lower intercorrelation with all other variables equal.

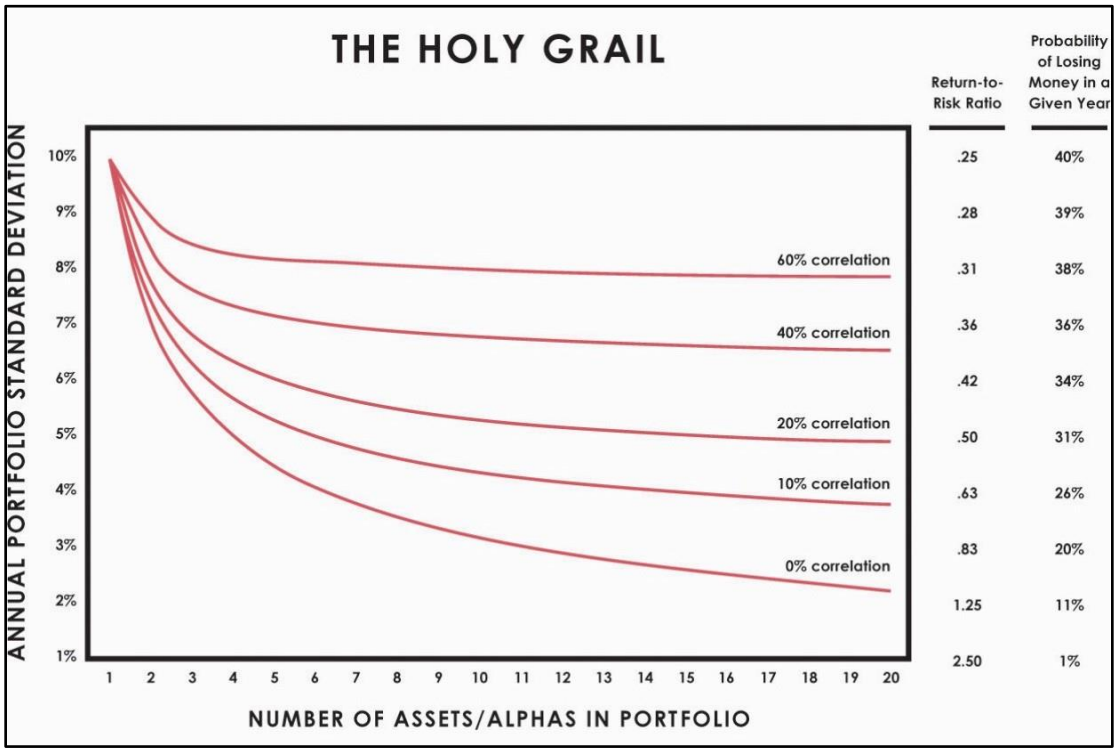

Figure 1 - The Holy Grail of Investing (Dalio, 2017)

Math of Ensembles

A core concept in machine learning is the bias-variance trade off. A model with high variance does not deliver consistent predictive accuracy for different datasets. A high variance model is said to be overfit: the model performs exceptionally well on the training set but does not generalize well out of sample. Overly complex models overfit the data because they learn spurious relationships within the training set which do not generalize into the out of sample data (Overing, 2019).

A high bias model is said to be underfit: the model performs poorly across training and test sets. An insufficiently complex model will underfit: relationships within the dataset will not be learned in either the training or test sets. Bias and variance generally have a negative relationship: as one is decreased, the other increases. When developing predictive models, both bias and variance must be balanced to make accurate predictions out of sample.

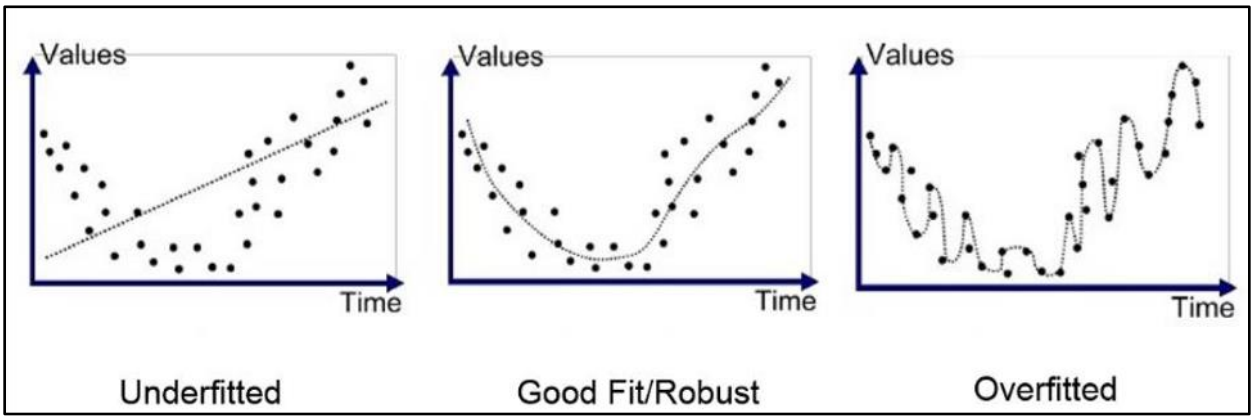

Figure 2 - Bias-Variance Trade-off (Bhande, 2016)

Ensembles are independent predictive models which are fused to achieve higher predictive power than any individual model on its own. Using ensembles is one of the few ways to reduce bias without increasing variance (Overing, 2019). Each predictive model optimizes differently and subsequently has model specific bias. Since each model has its own bias, using a combination of models reduces the collective bias by partially neutralizing each model's

idiosyncratic bias (Panchekha et. al, 2018). This process resembles the idiosyncratic risk reduction of diversification in portfolio theory.

Benefits of using ensembles can be demonstrated using probability theory. If two uncorrelated predictive models which each have an accuracy of 60% and are in consensus, the probability of a correct classification jumps from 60% to 84% (Panchekha et. al, 2018). Under the same circumstances, if 5 uncorrelated predictive models are in consensus, the probability of success increase from 60% independently to 99% collectively. While the results are staggering in theory, finding 5 truly orthogonal predictors is next to impossible. Ensembles perform better when their constituent predictions are lowly correlated. For this reason, using a wide range of different predictors will lead to better ensemble prediction accuracy.

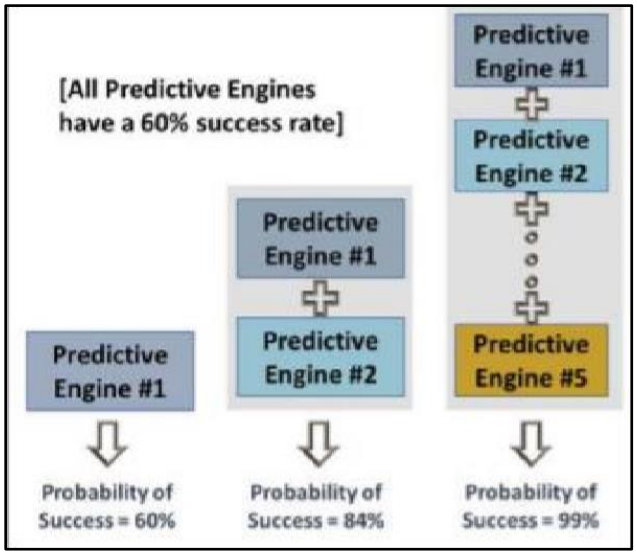

Figure 3 - Ensemble Predictive Gains (Panchekha et al., 2018)

Hyperparameter Selection Methods

The easiest way to set machine learning hyperparameters is to test them by hand. This involves using human judgement along with trial and error to guess which hyperparameter sets might be adequate. This method can be practical for algorithms which are not sensitive to their hyperparameters or algorithms with very few hyperparameters such as regularized linear

regression. More powerful algorithms tend to have more hyperparameters: cutting edge machine learning models such as deep neural networks have more than 10 highly significant hyperparameters, making them nearly unusable without hyperparameter optimization. Testing machine learning hyperparameters by hand is very inefficient: even if an algorithm has a small number of hyperparameters, using hyperparameter optimization will still find better solutions faster.

Unguided search involves optimizing an objective in an arbitrary way. An unguided search for hyperparameters tests discrete sets of hyperparameters and evaluates their efficacy. The most fit of the arbitrarily tested models is selected as the working model. A grid search is when the user defines equally spaced model parameters and each combination is tested. The computation time of a grid search increases exponentially as the number of test points and hyperparameters increase. Unguided searches test equally across the optimization manifold with much computation time being wasted on unpromising regions of the search space. Unguided searches are still more efficient than tuning models by hand because the computer can test the fitness of multiple parameter sets much faster than a person could change the parameters by hand and manually execute the program again.

The computational waste of hand tuning and unguided searches motivates this paper to use a better method: guided searches. Guided searches focus their computing power on the search subspaces which are most likely to contain unfound optima (Marques, 2006). Just as importantly, guided searches spend less computation power on regions of the objective manifold which likely do not contain optima. Not only are guided searches more efficient than unguided searches, they often find better solutions (Overing, 2019). Since unguided searches arbitrarily test parameter sets, the parameters they evaluate are likely not the global optimum. Guided

searches on the other hand, actively seeks out global optima. The only case where unguided searches would be more efficient than guided searches is when the marginal guided search computational overhead exceeds the marginal computational overhead of evaluating a point on the objective surface. Guided searching is still often more effective in this case as it views the objective surface as being continuous unlike unguided search which views the search space as being discrete. The continuous nature of guided searches allows it to test regions of the search space that unguided search would not be able to see.

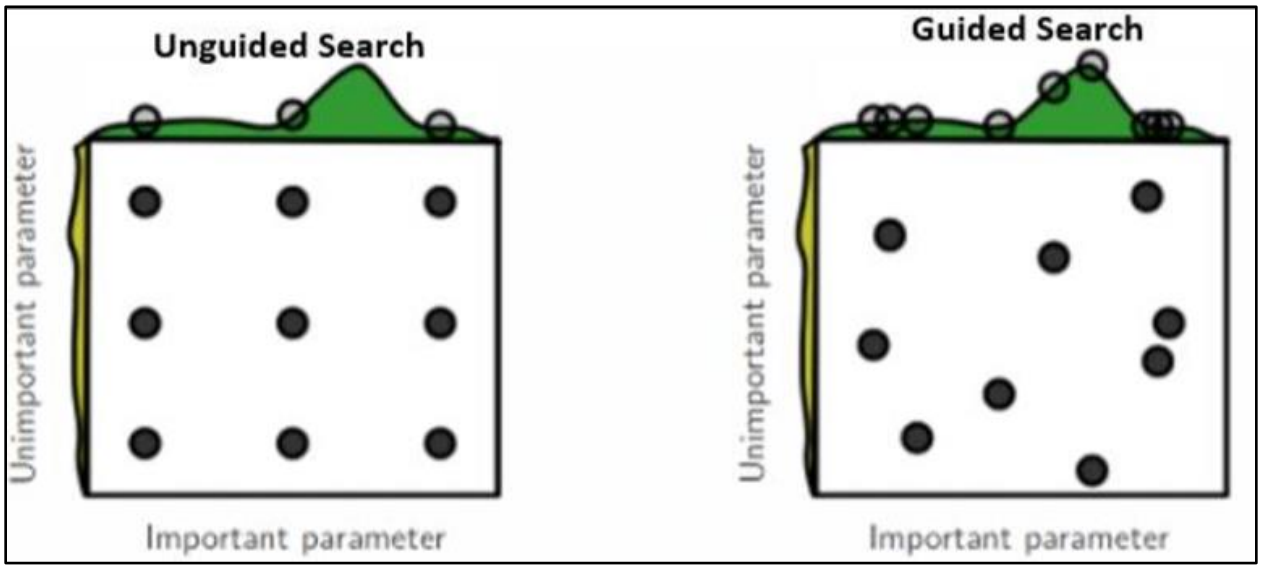

Figure 4 - Unguided Search Vs. Guided Search (Paul, 2018)

The algorithm proposed in this project fits into the study field of metaheuristics (Overing, 2019). A significant amount of metaheuristics research has focused on finding the optimal hyperparameters for single models. There are very few research papers pertaining to finding optimal ensembles of machine learning models. This paper has a high degree of originality: papers of a similar nature tend to focus on feature selection ensembles or ensembles of a single type of machine learning model. Instead, this paper solves for the optimal ensemble of multiple machine learning or financial models.

Population Based Training of Neural Networks

DeepMind is a firm that became world renowned for beating the world champion of Go with an artificial intelligence; a job earlier believed to be impossible with existing computing power (AlphaGo, 2017). DeepMind developed a genetic programming technique called population-based training (PBT) to discover effective neural network hyperparameters (Jaderberg et. al, 2017). In PBT, unsuccessful solutions randomly utilize parameter values from the top-performing models (Overing, 2019). The DeepMind Team uses genetic algorithms instead of hand tuning their models or searching randomly because they find appropriate hyperparameters quicker and devote more search time to the most promising hyperparameter settings.

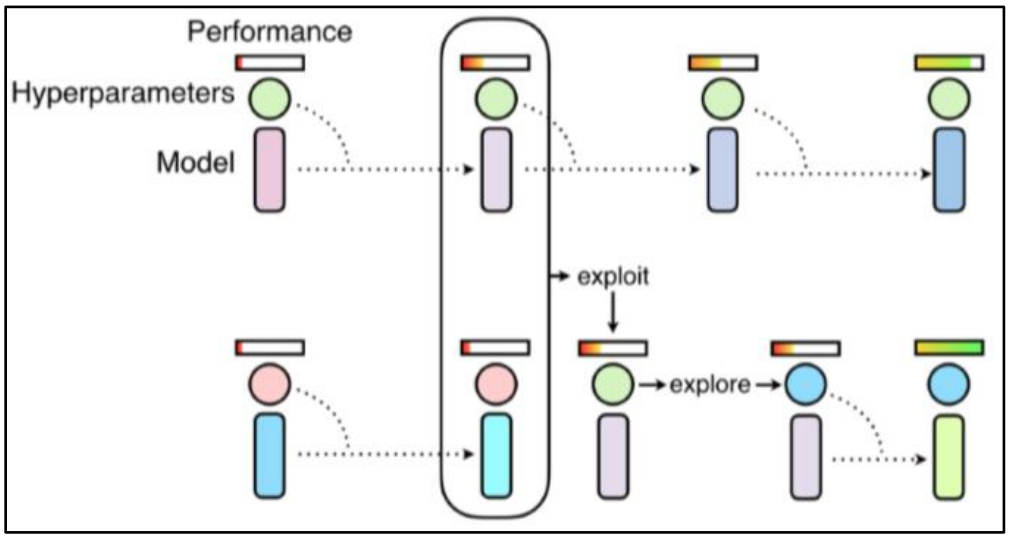

Figure 5 - Benefits of Using Genetic Algorithms (Jaderberg et. al, 2017)

Using PBT, DeepMind exceeded the performance of cutting-edge algorithms for a wide range of use cases (Jaderberg et. al, 2017). In Figure 6, PBT is shown to outperform multiple best in class machine learning solutions in fields such as language translation and video game bot performance.

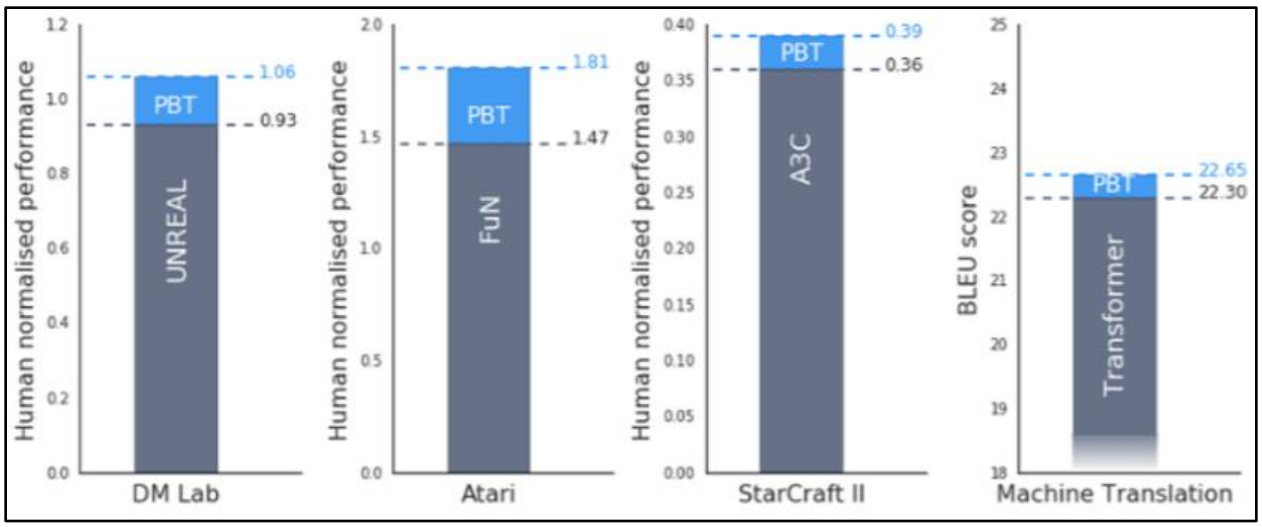

Figure 6 - DeepMind's PBT Performance (Jaderberg et. al, 2017)

For the purposes of this paper, the success of DeepMind using genetic algorithms to optimize hyperparameters is promising. DeepMind's PBT differs greatly from this project because while DeepMind's PBT searches for a single optimal model, this paper is aims to find the optimal model set (Overing, 2019).

Evolutionary Ensembles: Combining Learning Agents using Genetic Algorithms

In 2005, Notre Dame University academics released a paper on the use of evolutionary algorithms to generate machine learning ensembles. Their strategy to optimizing ensemble models was to use embedded genetic algorithms (a genetic algorithm operating within another genetic algorithm) (Sylvester & Chawla, 2005). The internal GA resides within the external GA's fitness function. The internal GA contains a population of distinctive machine learning models each with unique hyperparameters. This internal GA searches for the machine learning model which maximizes cross validation accuracy. The external model was dubbed “EVolutionary ENsembles" (EVEN). EVEN weights each agent in the population’s predictions and like the internal model, aims to minimize cross validation error.

The researchers use four distinctive datasets to validate their outer model's efficacy against an equal weighting of the internal model's population of predictions (Sylvester & Chawla, 2005). The paper's authors found that the EVEN model outperformed the simple average of the inner model's population (Overing, 2019). Another benchmark that was used was the uniform vote of each of the inner algorithms' binary classification. The EVEN algorithm only slightly exceeded the performance of using the classification value of each individual agent as a vote. The power of ensembles is shown by the fact that the novel EVEN algorithm and the uniform vote method were far more accurate than an average of the internal model's solutions. Since the difference in performance of EVEN and uniform vote are statistically insignificant and EVEN is massively more computationally inefficient than uniform vote, using uniform vote is suggested to be a superior method.

	Satimage	DNA	Phoneme	Pendigits
Best	88.57	94.97	90.99	99.33
Average	76.52	85.93	81.34	80.48
Uniform Vote	88.54	94.72	85.33	98.48
EVEN	89.94	94.97	89.77	99.21

Figure 7 - Evaluating EVEN's Classification Accuracy (in %) (Sylvester & Chawla, 2005)

A criticism of this publication is that the authors concentrated primarily on the development of the external GA and neglected the development of the internal GA. The inner GA tests the classification accuracy of each model and hyperparameter set individually. The problem with using a vanilla GA to test the models individually is that populations will likely end up being homogeneous as the GAs continue crossbreeding. The fact that the EVEN algorithm and uniform voting system are statistically indistinguishable suggests homogeneity within the internal GA population as an optimally weighted set of models showed no difference

from a uniform voting system. Ensembles thrive when their constituent models are diverse. The researches would have benefited from more time and thought to the development of the inner GA which chooses which machine learning models are represented. By considering the interrelationships between the predictions made within the inner GA, the researchers likely could have attained higher out of sample prediction accuracy. Sylvester & Chawla could have improved the evaluation of their algorithm by using an equal intensity unguided search which would test whether the extra computation used to run the genetic algorithm had merit.

This 2005 paper is the most similar paper to this one. The papers are similar in that they both use genetic algorithms to select from a number of different of machine learning algorithms. The papers are very different in terms of how the genetic algorithms are used: they focused a bulk of the computation on deciding how much to weight each machine learning model. In this paper, a vast majority of the computation is dedicated to searching for the optimal set of models instead of how much to weight each model. Another difference between the two papers is that this paper does not define the number of machine learning models that will be included in the final optimal ensemble. Instead, the genetic algorithm keeps adding additional models to the ensemble or terminates after a user defined number of generations. The critiques of the Evolutionary Ensembles paper have motivated innovations for this project such as optimizing upon the entire ensemble's fitness instead of the fitness of individual models. This fix will help tackle the problem of population homogeneity which seems to have occurred in the Notre Dame researchers' paper.

Method

The Generalized Ensemble Optimizer follows three steps. The first step is to find the optimal hyperparameters for each model individually. The second step is to fuse each individual

model's outputs into a single ensemble. The final step is to use a genetic algorithm to find additional models which make positive marginal contributions to the ensemble model.

Descriptions of Algorithms Used

Ensembles work best with diverse models because each model exploits different types of relationships between variables and will be better suited to correct each others' inaccuracies. The ensembles developed in this project use three distinctive algorithms to perform classification: Extreme Gradient Boosted Decision Trees (XGBoost Decision Trees), a Stochastic Gradient Descent Logistic Classifier (SGD Classifer) and Support Vector Machine Classifier (SVM Classifer). Python was used to prototype and test the novel algorithm. This algorithm was coded in a robust manner which allows the constituent models, objective function and dataset to be easily swapped out for any other model that currently exists or will exist.

Hyperparameters to Be Optimized							
XGB	Estimators	Eta	Depth	Min Child Weight	Gamma	Subsample	Col Subsample
SGD	Iterations	Alpha					
SVM	Iterations	C					

Table 1 - Optimized Hyperparameters by ML Algorithm Used

XGBoost Decision Trees

XGBoost is a very fast and powerful algorithm which ensembles decision trees in an ingenious way. Classification and Regression Trees (CARTs) supervised learn the optimal set of decision rules and weights for a given training set (Chen, 2014). At each split of the tree, a real value bisects a single feature in the dataset, sorting each datapoint according to the decision rule. For example, in "tree1" in Figure 8, the real value 20 bisects the feature "age," so all datapoints with age less than 20 are sorted into one group and vice versa. Like a tree trunk splits into branches, the tree splits decisions further and further. At the bottom of all splits is a weight value: how much to weight data samples which follow the decision criteria leading to that node.

Singular decision trees massively overfit because they perfectly learn the decision rules and weights of the training set with no regularization.

CART tree ensemble methods were developed to overcome the overfitting problem of singular CARTs (Craib, 2016). The random forest algorithm was developed which randomly subsets each trees' features and ensembles the randomly subset trees (Chen, 2014). After the trees are randomly subset and optimized the sum of their prediction values are used as the random forest prediction as seen Figure 8. Ensembles where each constituent model uses a random subset of data is known as bootstrapping.

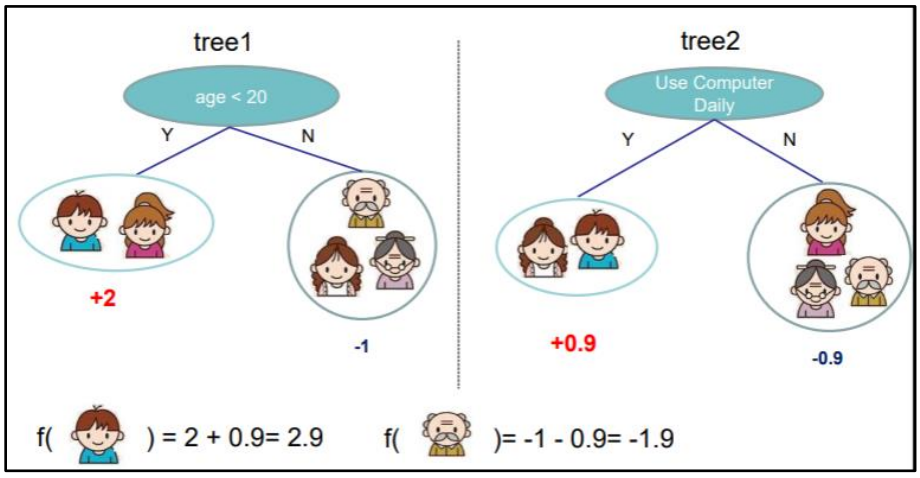

Figure 8 – Basic Tree Ensemble (Chen, 2014)

Gradient Boosting Machine Trees (GBM Trees) were later developed to take the randomness out of growing the ensemble of trees (Craib, 2016). Boosting finds the optimal additional tree by minimizing the joint error of the current ensemble and the next tree (Chen, 2014). Boosted trees aim to correct the errors of the existing ensemble of trees. Stochastic gradient descent cannot be used to find the optimal additional tree because joint loss cannot be directly represented in vector form (Chen, 2014). Instead additive training is used by optimizing the Taylor expansion of the sum of the loss functions of each tree. This process resembles the method used in this project to find the next algorithm to add to the ensemble; in the final step, a

genetic algorithm searches for the next model which increases marginal predictive accuracy the most. Unlike decision tree optimization, machine learning cannot be used to optimize an ensemble of hyperparameters because the search space is undifferentiable. GBM was implemented with no regularization term, making it prone to overfitting.

XGBoost was developed in 2014 to overcome GBM's short comings such as long computation time and lack of regularization. XGBoost is a cutting-edge tree ensemble model, using gradient boosting and regularization methods such as tree-pruning to avoid overfitting and eliminate bias. XGBoost has a high number of hyperparameters and it is very easy to overfit if they are not tuned correctly.

In GBM, each tree is grown to a user defined maximum depth (Chen, 2014). The higher the max tree depth, the more likely the model is to overfit because each tree has more decision splits, thus adding often unnecessary complexity to the model. XGBoost also has the hyperparameter max depth but the model does not have the same tendency to overfit for high max tree depths because XGBoost prunes its trees. On every iteration, XGBoost grows its tree all the way to the maximum depth just like in GBM and then it prunes all solutions which have cumulative weights less than the minimum child weight hyperparameter (Morde, 2019). This prevents trees from overfitting because tree rules will not be made unless they pass the significance threshold of minimum child weight. An additional regularization term is gamma: additional decision splits will not be made unless the tree's loss is reduced by a value greater than gamma (XGBoost Documentation, n.d.). The learning rate (eta) is another hyperparameter whose tuning helps model performance. Learning rate defines how much weights are updated each iteration in gradient descent (Zulkifli, 2018). If a learning rate is too small, gradient descent will be slow to converge because it takes far more iterations before reaching the global optima.

Learning rates set to be too small can also fail to converge as they can get stuck into a local optimum. If the learning rate is too large, the model will fail to converge because it will jump over the global optima. Subsample is the number of learning samples each tree uses to fit. This is a bootstrapping method which prevents overfitting by only allowing each tree to use a certain amount of data to optimize. Column sample by tree is the percentage of features which each tree subsets also in a bootstrapping manner. Number of estimators defines how many trees are used in the ensemble. If the number of trees is set to be too high, the final marginal trees will exploit spurious relationships in the dataset and overfit. If this value is set to be too low, the model will leave important relationships unexploited and can lead to underfitting.

Not only is XGBoost more powerful than its preceding tree methods, it also is faster (Chen, 2014). By leveraging multicore computing and efficient optimization approximation methods, XGBoost runs extremely efficiently.

As with most powerful machine learning algorithms, it is extremely easy to overfit XGBoost models, so adept parameter tuning is critical. The hyperparameter space for XGBoost is massive: in this project the 7 significant hyperparameters named above are tuned. The *XGBClassifier* function from Python's *xgboost* library was used to implement the XGBoost algorithm (XGBoost Documentation, n.d.).

Support Vector Machine Classifier (SVM Classifer)

Support Vector Machine Binary classifiers (SVM classifiers) separate points in multidimensional space and labels them as belonging to one class or another (Patel, 2017). When using linear SVMs for binary classification, the SVM maximizes the distance from the hyperplane to each point (Chen, 2019). A hyperplane is a n-1-dimensional subspace within a n-dimensional space. For example, on a 2-dimentional graph, a hyperplane would be 1-dimentional – a line. Since linear SVMs find the hyperplane which maximizes the distance to each point, a

single outlier can fundamentally change the optimal hyperplane. After fitting the model, future data vectors are classified according to which side of the hyperplane each set of features lies upon. Since outliers can immensely skew the optimal hyperplane, vanilla SVMs are extremely prone to overfitting.

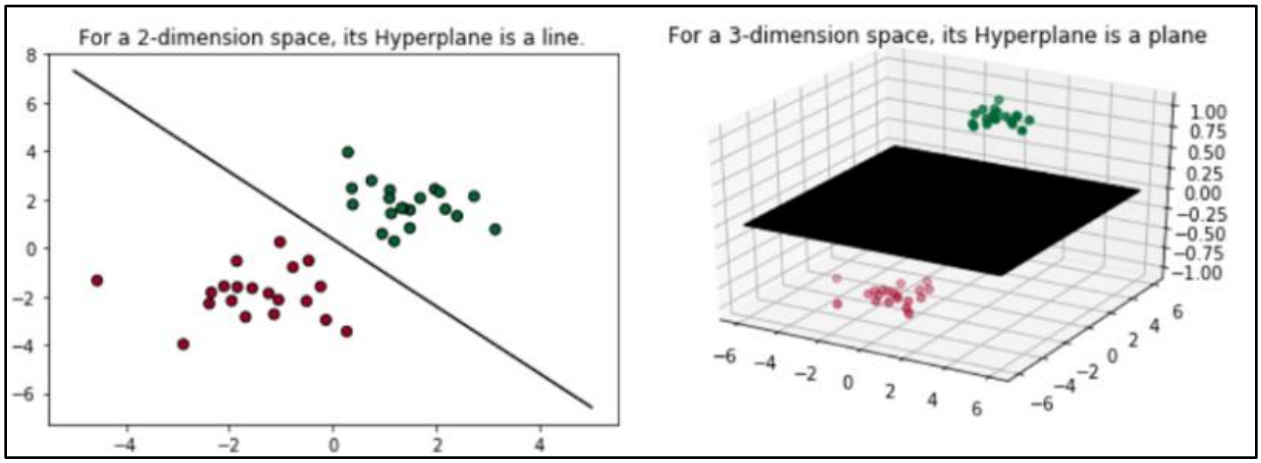

Figure 9 – Hyperplane in 2D and 3D (Chen, 2019)

Most datasets are not linearly separable as seen in the figure above and if vanilla SVMs are used as described above, the model is very likely to overfit. Soft margin SVMs generalize far better for not linearly separable datasets (Chen, 2019). Instead of only optimizing for the hyperplane with the largest minimum margin, soft margin SVMs simultaneously solve for the hyperplane which correctly separates the most points. The hyperparameter C dictates the trade-off between highest in sample classification accuracy versus the closest point to the hyperplane's maximum distance (C-Support Vector Classification, n.d.). Too large values of C severely penalize misclassification and will lead to overfit models that are heavily influenced by outliers. Too low values of C neglect outliers and can produce underfit models which misclassify entire clusters of data. C and number of iterations are the two hyperparameters which are optimized in this project. Number of iterations are optimized so that SVM can benefit from early stopping and prevent overfitting. The function *svm.SVC* was used from Python's Sklearn package.

Stochastic Gradient Descent Classifier (SGD Classifer)

Stochastic Gradient Descent Classifiers are a set of regularized linear models which use stochastic gradient descent to efficiently classify. Regularized logistic regression optimized by stochastic gradient decent is the regularized linear model used in this project. Logistic regression filters its input through a sigmoid function which asymptotically bounds the input values between 0 and 1 (Google Developers, 2018). The asymptotic bound is extremely useful for computing probabilities because out of sample values which are outside the range of training datasets' values will not return incomprehensible probabilities outside of [0,1]. Logistic regression uses log-loss instead of linear regressions' mean squared error because it is a better surrogate function for binary loss (Google Developers, 2018).

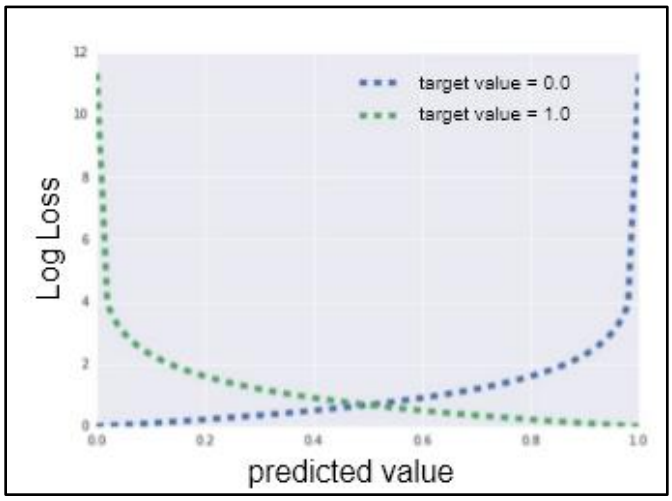

Figure 10 – Asymptotic Bounds of Logistic Regression (Google Developers, 2018)

Vanilla logistic regression is prone to overfitting because the algorithm will attempt to push the log-loss further along the asymptote to zero (Google Developers, 2018). L2 regularization and early stopping can be used to avoid overfitting. L2 regularization penalizes the magnitude of weights on each feature which prevents the algorithm from learning spurious relationships which exist only within the training set (Nagpal, 2017). Early stopping is a regularization method which prevents overfitting of iterative learners, such as gradient descent.

Stopping training before an algorithm begins to over-train can increase the generalizability of the model.

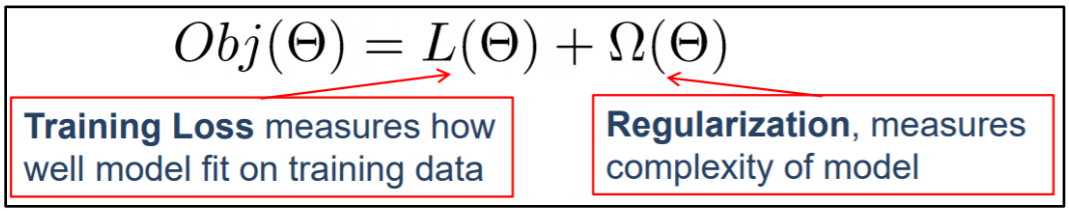

Figure 11 – Regularized Objective Function (Google Developers, 2018)

The SGD Classifier's hyperparameter values for alpha and number of epochs can fundamentally change the output of the model. The hyper parameter alpha dictates the weight to put on the L2 regularization term. If alpha is set to 0, vanilla logistic regression will be performed, and the algorithm will very likely overfit. If alpha is set too high, model weights will be too heavily penalized which would lead to the model underfitting. Finding a suitable alpha value is essential to creating a generalizable model (Nagpal, 2017). By tuning the number of epochs, early stopping can be used to create more generalizable models (Google Developers, 2018). If the number of epochs is set too low, the model will not converge and thus the solution will underfit. By setting the number of epochs value too high, the model will not take advantage of early stopping and wastes computation time. The values for alpha and number of epochs are the hyperparameters which are optimized in this project. The SGDClassifierfunction from Sklearn was used to implement this algorithm.

The starting point of the Generalized Ensemble Optimization is to find the optimal set of parameters for each machine learning model individually. This set of individually optimized algorithms will form the foundation of the ensemble and all subsequent steps aim to improve upon this foundation. There are a plethora of existing methods for finding the optimal hyperparameters of a single model. Using guided search algorithms such as Bayesian

optimization reduces the amount of computation required to find an optimal set of hyperparameters versus unguided search methods such as grid or random search.

Bayesian Optimization & Expected Improvement

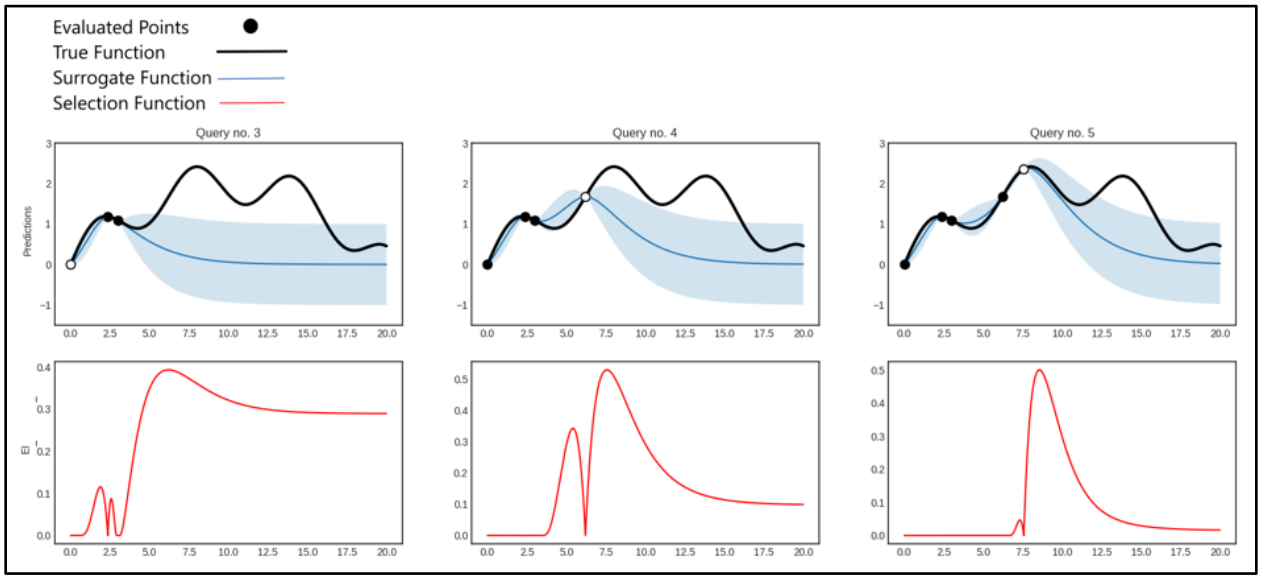

Figure 12 – Three Iterations of Bayesian Optimization (Kraus, 2019)

Bayesian optimization (BO) uses prior evaluations of the objective function to decide where on the optimization manifold to search next (Kraus, 2019). Bayesian optimization focuses its computation on the most promising areas of the search space and disregards areas of the search space which aren't expected to improve the model. In this case, the search space is the set of hyperparameters for each individual predictive model. The objective function used in this project is the cross-validation binary accuracy of each model. As more prediction accuracies are evaluated for different sets of hyperparameters, a surrogate function is able to make a better approximation of the shape of the true objective function. A selection function is then used to select the point on the surrogate function which is most likely to be useful for searching the parameter space (Kraus, 2019). The selection function used for this algorithm is the Expected Improvement metric. Expected Improvement has an embedded trade-off between exploitation and exploration: it searches for both solutions which are expected to have the highest fitness

values as well as searches the least explored regions of the objective space. This Bayesian optimization model repeats the process of updating the surrogate and selection functions for a user defined number of iterations. Finally, when the set number of iterations are exhausted, the optimization returns the set of hyperparameters with the highest out of cross-validation predictive accuracy.

In the second step, a blending model is used to ensemble the models which were optimized in step 1. Blending models use the output of a collection of models as an input and create a meta-model which is at least as strong as the strongest input model. Uniform vote ensembles as described in Sylvester & Chawla's Evolutionary Ensemble paper are used for the blending model in the tests run in this paper (Sylvester & Chawla, 2005). Other blending models such as sigmoid averaging, equal weighted predictions or fitting an optimization model such as logistic regression or neural networks could also be used.

Steps 1 and 2 initialize the ensemble with a reasonable basis for the optimal ensemble and step three improves the ensemble further. In the final step, a genetic algorithm aims to find models which further improve the ensembled model found in step 2. While the bulk of the innovation of this project exists in step 3, steps 1 and 2 are necessary to avoid blindly searching among an infinite optimization manifold.

Genetic Algorithms

Genetic algorithms are a guided search optimization method which takes inspiration from the way organisms evolve in nature. Using concepts such as survival of the fittest and genetic mutation, genetic algorithms evolve candidate solutions to become increasingly fit. Genetic algorithms operate using four iterative steps: spawning, selection, crossbreeding and mutation. In the spawning phase, random solutions within the search space are spawned (Overing, 2019). The set of solutions is called the population and each individual solution is known as an agent. The

GA then evaluates each agent's fitness: how well does each solution score on the objective function. Agents with the highest fitness are selected and are kept in the population while the rest of the solutions are deleted. The selection process is inspired by survival of fittest: agents with the highest fitness survive while the rest of the population dies. After selection, the surviving solutions crossbreed: inputs from two or more solutions are combined to create new solutions. The crossbreeding phase is analogous to natural selection – the fittest solutions of the population interbreed, likely increasing the average fitness of the population. Finally, each solution has a user defined probability to be mutated: random noise is applied to the solutions' inputs. Mutation allows the genetic algorithm to search uncharted territory of the objective manifold.

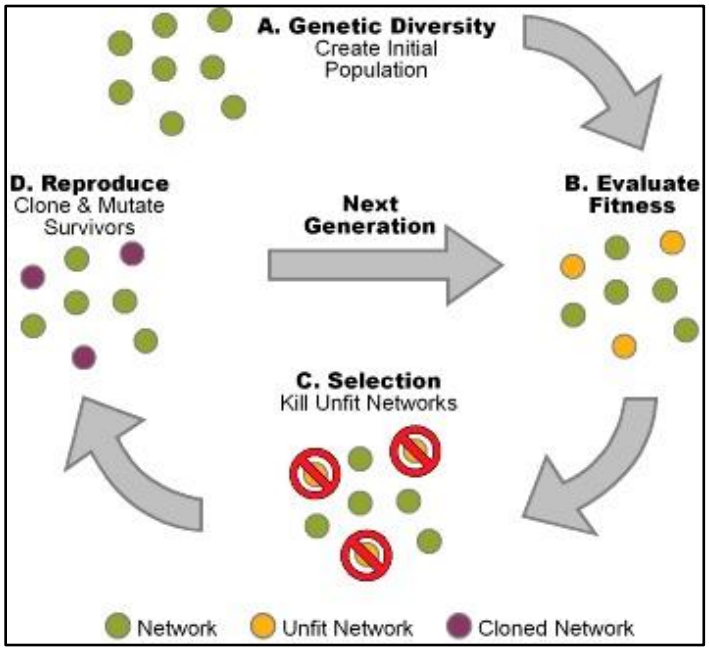

Figure 13 - Genetic Algorithm Visualized (Applying Genetic Algorithms, 2018)

Genetic algorithms' results are generally not very sensitive to their hyperparameters, but their tuning can make convergence more likely and efficient. Population size dictates the number of random solutions which are spawned in the beginning and respawned in future generations. Number of generations is the number of times the algorithm iteratively kills and respawns

agents. Number of generations multiplied by population size is the total number of fitness function evaluations that must be performed. The higher the population and number of generations, the more likely the algorithm is to converge but at the cost of a higher computation time. Probability of crossover is the likelihood that any two randomly selected surviving agents breed. The higher the probability crossover, the more the algorithm will exploit regions of the manifold which are known to have high fitness. The lower the probability of crossover, the more the algorithm will explore random regions of the search manifold. Probability of mutation is the likelihood that an agent's genes will be mutated. High values of probability of mutation explore the manifold more while low values exploit known fit solutions more often.

Genetic algorithms are built to find the global optimum of any objective function and do this very well. GAs aim to perfectly optimize any given fitness function without taking the shape of the optimization manifold into account. Machine learning on the other hand measures the gradient (and often the hessian) of the objective function and moves toward the steepest gradient. The main difference between machine learning and other optimization methods such as genetic algorithms is that machine learning iteratively learns from the shape of the optimization manifold while other optimization methods do not. Since genetic algorithms aim to find the global optimum and do not take gradient into account in any way, they are very prone to overfitting. For this reason, genetic algorithms are not a good substitute for machine learning methods for most use cases. Genetic algorithms also have no built-in method for regularization: the fitness function defined by the developer will be explicitly optimized. If regularization is required, the developer must include it in the fitness function. The main advantage of using genetic algorithms over machine learning methods is that the fitness function does not need to be differentiable. Since the gradient and hessian of the fitness function do not need to be taken, the genetic algorithm can

optimize any function including discrete or discontinuous functions. Since hyperparameter search has a disjoint and often discontinuous optimization manifold, machine learning cannot be used and other optimization methods such as Bayesian optimization and genetic algorithms must be used instead.

Genetic algorithms are adept at adapting to shifting optimization manifolds because they evolve as the objective space shifts. The adaptive nature of genetic algorithms is its main advantage against other guided search methods such as Bayesian optimization which needs to start anew every time the optimization manifold shifts.

One problem with ensembles of genetic algorithms populations is homogeneity of the population's solutions. As the most fit solutions continue to survive, they have the highest chance to crossbreed with other solutions in every generation. As the number of generations increases, the most fit solutions continue to interbreed. This often leads to the single most fit solution dominating the gene pool; all solutions in the population are just slight variations of each other. Ordinarily this is not a problem, as genetic algorithms are built to find the single optimal solution. The ensemble created in this project lies outside of the genetic algorithm which avoids the population homogeneity problem.

The fitness of each solution is evaluated as its marginal contribution to the ensemble: the accuracy of the ensemble including the proposed solution minus its accuracy without the proposed solution. Every time a new solution is inducted into the ensemble, the optimization manifold shifts. The constantly evolving objective manifold is why genetic algorithms are well suited to finding the optimal ensemble versus other optimization methods.

Evaluation

Numerai Dataset

The dataset used to test the Generalized Ensemble Optimizer algorithm is a Numerai competition dataset. Numerai is a hedge fund based out of San Francisco whose description seamlessly involves about every modern tech buzzword. Numerai crowdsources machine learning models for quantitative trading using cryptocurrencies as incentivizes (Craib, 2016). Hedge funds typically aim to find market edges then protect their edges so other funds won't front-run the edges. In quantitative funds, alpha can be derived from proprietary datasets, so funds aim to monopolize them. Richard Craib, founder of Numerai, was employed by one of such funds (Craib, 2018). He built a methodology which converts financial data into a binary classification problem. Craib used machine learning to classify the data and deployed his strategy onto $50 million USD of investors' capital (Craib, 2018). The strategy performed well but Craib believed that through crowd sourcing, an ensemble of machine learning models could outperform the model he developed.

Craib was inspired by Yann LeCun who in 1998 made the MNIST dataset open source and encouraged researches to experiment with it (Craib, 2016). The MNIST dataset contains a collection of images of handwritten numbers and the true intended value of each handwritten number. By 2012, a huge ensemble of machine learning models could identify handwritten digits with approximately human precision.

Craib began thinking of ways to crowdsource an edge in financial markets without giving competitors their proprietary data for free (Craib, 2016). To crowdsource his hedge fund, Craib used homomorphic encryption. Homomorphic encryption performs high level algebraic operations to the dataset, so it is unidentifiable but still preserves its structure. Unlike classic encryption, machine learning models trained on homomorphic encrypted data are still useful on

the unencrypted financial data. Numerai gives its financial homomorphic encrypted dataset away for free. Users do not even need to create an account or even speak English to download the dataset. Numerai uses an ensemble of crowdsourced machine learning models' outputs to trade (How Numerai Works, 2019). The Numerai dataset is great for computational finance academics because it provides a clean and open sourced dataset for testing new ideas. The dataset is big: there's 50 unique features, over half a million training samples and over 70,000 training and test samples.

Numerai Competition

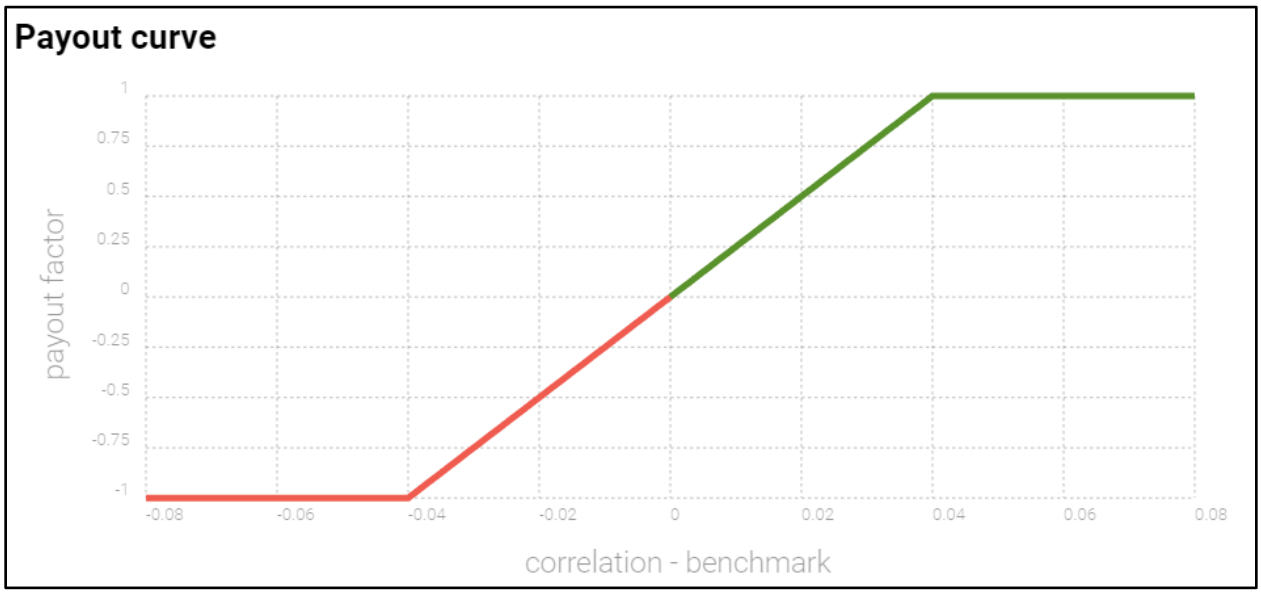

Figure 14 - Numerai Payout Curve, (How Numerai Works, 2019)

Every Saturday, a new unique dataset is released containing the encrypted and regularized financial data for training (How Numerai Works, 2019). For the rest of that weekend, users can stake Numeraire. Numeraire is a cryptocurrency created by Numerai to incentivize competition participants to make accurate predictions. When staking, Numerai competition participants enter their percentage confidence of their model and lock in any amount of Numeraire for a month. Models with the highest confidence are sequentially accepted into the

competition until 2000 Numeraire has been staked. The minimum confidence percentage accepted into the staking pool is set as the competitions' benchmark. After a month passes, the amount of Numeraire returned to each participant is dictated by their model's performance relative to the benchmark. The payout curve is a winsorized line. Each competitors' percentage return on staked Numeraire is their nominal correlation outperformance of the benchmark clipped at ± 100%. Numerai disincentives users from blindly swarming the competition with overfit or randomly generated solutions by making competition participants stake cryptocurrency to enter the competition. Since each participant has skin in the game, they have a vested interest in their predictions generalizing out of sample.

Equal Intensity Random Search

The bulk of the computation time in guided and unguided searches are spent on evaluating fitness functions for a vast majority of all use cases. In this project, machine learning models are trained and tested within the fitness function. The computation required to evaluate the fitness function heavily overweighs the computation required to perform guided search. None the less, guided searches require greater computational overhead to compute which solution to evaluate next versus unguided searches. To justify the additional computation to guide the search, Chen & Navet created a test named equal intensity random search (Chen & Navet, 2007). In the paper "Pretests for Genetic Programing Evolved Trading Programs," Chen & Navet explore whether genetic algorithms can find viable systematic trading rules. To test the viability of their GA evolved trading rules, they tested its out of sample performance versus equal intensity random search (EIRS). Performing an equal intensity random search involves evaluating the same number of fitness functions with randomly generated trading rules as the number of fitness function evaluations performed in the guided search. Chen & Navet then compare the guided search's result to the distribution of randomly generated trading rules. Their

null hypothesis is that there is nothing to be learned from the past or that the genetic algorithm strongly overfits the in-sample data. The alternate hypothesis is that there are pervasive relationships in the data that the genetic algorithm can find and profit from.

Another test Chen & Navet propose in their paper is running an equal intensity random search with a cross validation stage. This involves running the same number of randomly generated trading strategies then comparing the genetic programming evolved trading strategies to the random strategies which performed the best in the cross-validation phase (Chen & Navet, 2007). This test was run to show whether using a cross-validation step increases the out of sample performance. Through their tests, Chen & Navet accepted the null hypothesis that genetic algorithms could not learn generalizable trading rules based on historical data.

Adaptations of Chen & Navet's rigorous tests are used to test the Generalized Ensemble Optimization framework. Equal intensity random search is used to spawn a population of randomly selected machine learning models with random hyperparameters. The Bayesian Optimization in step 1 uses 50 fitness function evaluations for each of the three models totaling 150 evaluations. The Genetic Algorithm in step 3 has 11 generations before converging and 30 in the population, requiring 330 fitness function evaluations. The equal intensity random search population will spawn a total of 480 machine learning models, equaling the total number of fitness function evaluations used for the Generalized Ensemble Optimization. The out of sample predictive accuracy of the Generalized Ensemble Optimization is compared to the distribution of out of sample predictive accuracy of the equal intensity random search.

To test the effectiveness of the cross-validation data, an equal intensity random search with a cross validation stage is run. In their paper, Chen & Navet do not specify the percentile

cut off they used to filter their cross-validation performance so the top 25, 10 and 5 percentiles are tested.

To test the effectiveness of unguided searches for optimal ensembles, ensembles are tested which combine the predictions of each of the top n-percentiles of the equal intensity random search will be tested. This test will reveal whether the Generalized Ensemble Optimization outperforms an ensemble of machine learning algorithms with randomly set hyperparameters. This test will also indicate whether there is a relationship between the size of the cross-validation ensemble filter or not.

Results

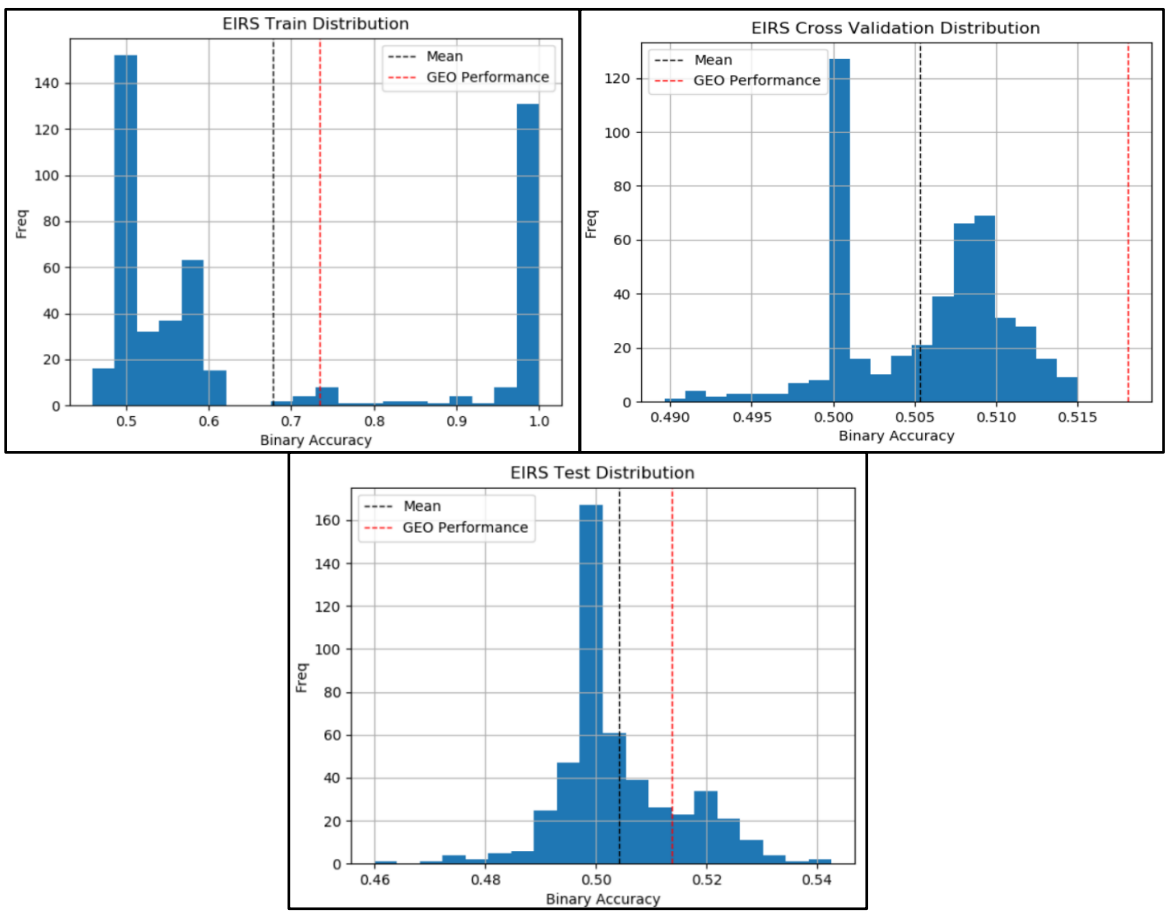

Figure 15 – Equal Intensity Random Search Binary Accuracy Distributions

The Generalized Ensemble Optimization's accuracy outperforms the average of equal intensity random search's binary accuracy across the train, cross validation and test samples. Training accuracy shows how well the in-sample data is being predicted by the machine learning algorithms. If training accuracies are irreplaceably high, the models are overfitting. If training accuracies are abysmally low, the models are underfitting. The equal intensity random search training performance shows a bimodal distribution with over a quarter of models massively overfitting with in-sample classification accuracies lying between 98%-100%. This result is intuitive because the machine learning algorithms used are all prone to overfitting without proper hyperparameter tuning. The second modal in the training accuracy distribution shows underfitting: approximately a third of the training prediction accuracies lie between 49% and 51%. This shows the waste of using unguided search methods: over half of the samples randomly spawned show signs of severe underfitting and overfitting. The Generalized Ensemble Optimization's training set performance is higher than the average of the equal intensity random search. The GEO's in-sample performance was 73.5% while the average of the EIRS was 67.8%.

The GEO's cross validation accuracy outperforms the entire distribution of EIRS cross validation accuracy with a 51.8% accuracy vs EIRS' average accuracy of 50.5%. Since the GEO explicitly maximizes the cross-validation accuracy using guided search methods, it is not surprising that the cross-validation accuracy is so high. This shows the power of ensembles: the GEO attained higher cross-validation accuracy than any individual model could.

The GEO attained a higher out of sample binary accuracy than the EIRS. The GEO's accuracy was 51.3% while the average EIRS accuracy was 50.4%. The equal intensity random search distribution of the out of sample data has far fatter tails than the cross-validation set. This is likely because Numerai's test set contains 1/30th of the samples as the cross-validation set,

making variance in the models' out of sample performance more likely. The GEO did not outperform the entire distribution of out of sample models as it did in the cross-validation set. This suggests that the GEO likely overfit the ensemble of hyperparameters to the cross-validation set. This type of overfitting is known as implicit overfitting: selecting models which worked well in the cross-validation set which do not generalize as well to the out of sample data. Despite implicitly overfitting, the GEO still performs better than the average of EIRS. It will be demonstrated in Figures 16 & 17 that the GEO implicitly overfits less than common model selection methods and that the right tail of the EIRS distribution which outperforms the GEO is not linearly separable using cross-validation

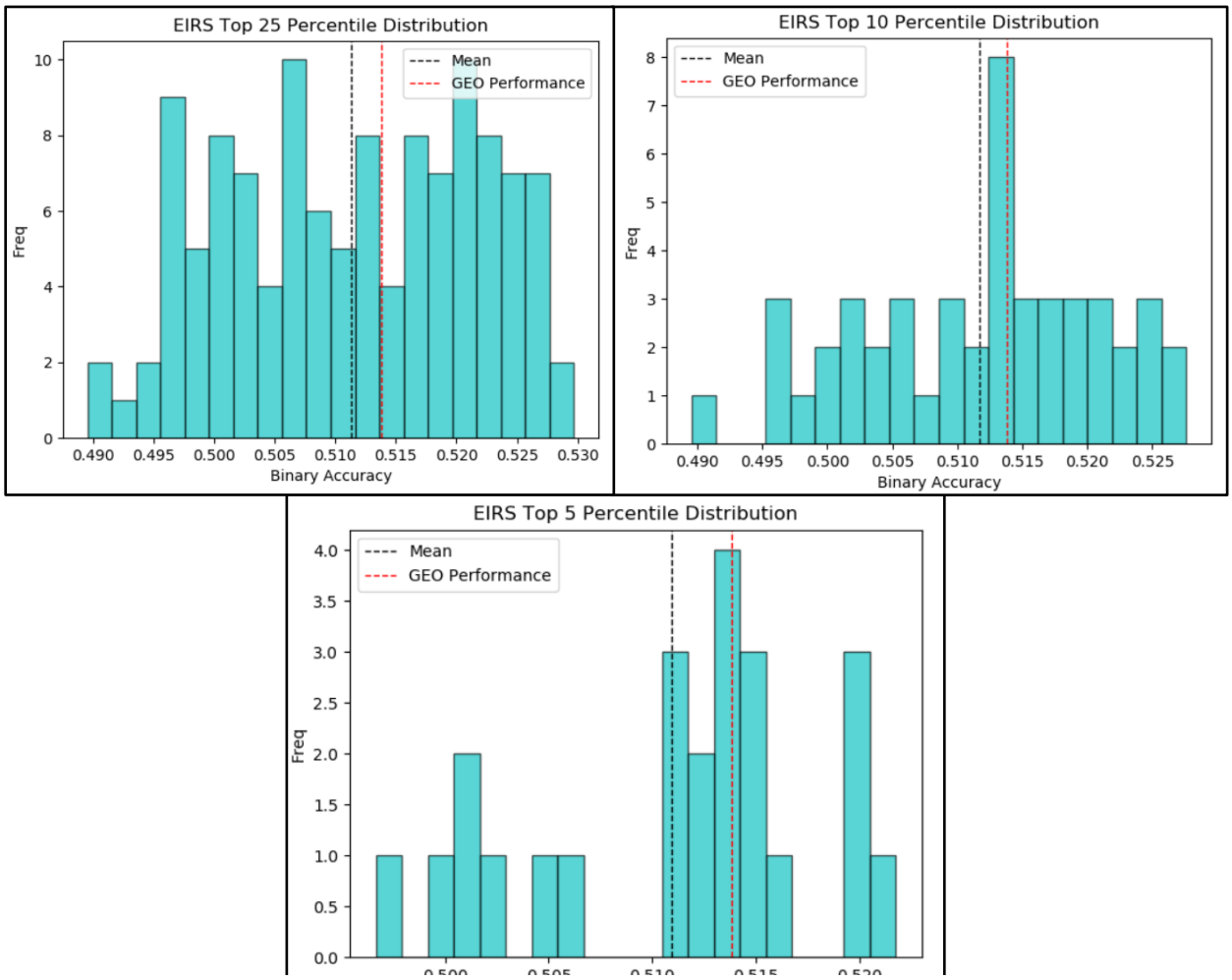

Figure 16 – EIRS Binary Accuracy Distributions of Top N-Percentile Cross Validation Accuracy

Figure 16 shows the distribution of out of sample accuracy for the EIRS models with the top 25, 10 and 5 percentile cross-validation accuracy. The visuals show that the GEO outperforms the average random model's accuracy for each percentile tested. This shows that models existing in the right tail of the EIRS's cross-validation distribution do not outperform the GEO on average. This indicates that although random models which outperform the GEO out of sample exist, they cannot be reliably subset using cross-validation, indicating a higher degree of implicit overfitting in EIRS cross-validation selection than with the GEO. The effectiveness of the cross-validation process is shown by all the right tail distributions' average test data accuracies are higher than the average out of sample accuracy of all the EIRS models combined with no cross validation step.

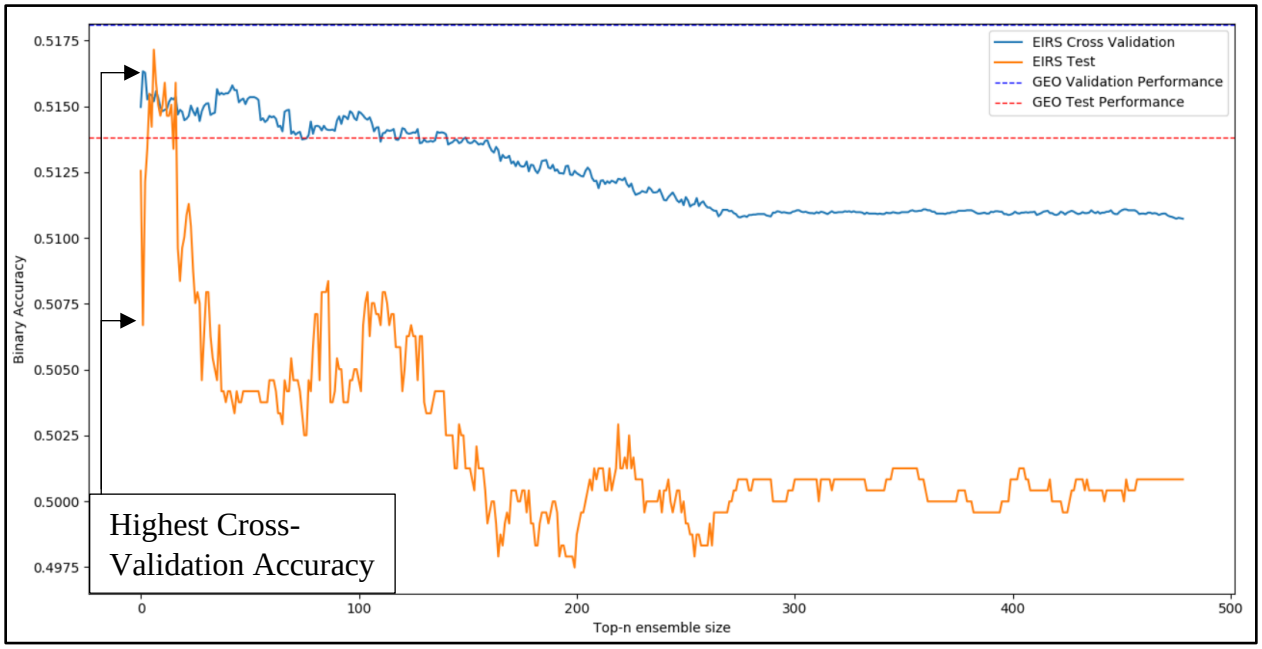

Figure 17 – Ensembles of EIRS Models by Cross-Validation Accuracy Ranking

Figure 17 depicts the cross-validation and test sample performance of ensembles made from the top n models ranked by cross-validation accuracy. The figure shows a general negative relationship between taking larger ensembles of top n performing models and cross-validation

and test accuracies. The accuracy then flatlines at around an ensemble of 275 models. The accuracies bottoming out resemble the asymptotes in Ray Dalio's "Holy Grail of Investing" as seen in Figure 1. The two graphs differ because Figure 1 shows portfolio volatility on the y-axis while Figure 17 shows model accuracy, which decreases as number of models increases. This suggests that the EIRS found a few models which generalize well while the rest severely underfit or overfit the dataset.

The bias introduced by selecting the top-n models by cross-validation accuracy can be seen by the ensembles' cross validation accuracies generally being larger than the test accuracy. Since the EIRS models' hyperparameters were not optimized in any way, the performance on the cross-validation and out of sample datasets of an ensemble of the entire EIRS model set are expected to be equal. Instead, the EIRS test accuracy flatlines far lower than the cross-validation set. The test data from a Numerai Competition contains $1/30^{th}$ of the number of data samples as the cross-validation set so discrepancies between where the cross-validation and test sets bottom out is likely a data artifact resulting from the smaller sample size.

Thus far, the results shown were inspired by Chen & Navet's equal intensity random search distributions as suggested in "Pretests for Genetic Programing Evolved Trading Programs." Since Chen & Navet were searching for singular models, testing their trading genetic algorithm's performance against randomly generated singular models was sufficient (Chen & Navet, 2007). Since the GEO algorithm searches for the optimal ensemble of models, its results should be tested against ensembles of randomly generated models.

The GEO's out of sample performance strongly outperforms all ensembles with 40 or more models. The EIRS found ensembles of models which outperform the GEO's performance out of sample. The ensembles of EIRS models which outperform the GEO out of sample cannot

reliably be found using the cross-validation set. The cross-validation accuracies of EIRS models show no differentiation between ensembles of models which outperform the GEO and those which underperform. The top-n ensemble ranked by cross-validation accuracy is the top-2 cross-validation accuracy model, attaining a cross-validation accuracy of 51.6%. The corresponding test accuracy of the best performing top-n ensemble is an abysmal 50.6% accuracy; more than a full percentage point drop from the cross-validation accuracy. This shows that the process of creating ensembles by cross-validation accuracy ranking implicitly overfits the cross-validation dataset more than the GEO algorithm does.

Discussion

Figures 15, 16 and 17 positively affirm the hypothesis that the Generalized Ensemble Optimization algorithm could outperform an equal intensity random search on average. Figure 15 shows that there is a drop off between cross-validation and test accuracy. This suggests that the GEO is likely implicitly overfitting the cross-validation set. Figures 16 and 17 show that the GEO implicitly overfits less than using a cross-validation filter on a universe of equal intensity random searched algorithms.

No feature selection or engineering was used in this implementation of the GEO. Many research papers have been published which automate the process of feature engineering/selection. Using feature engineering techniques along with the GEO would likely lead to performance gains of both.

To further test the GEO's efficacy in creating machine learning ensembles, other machine learning algorithms such as neural networks could be used as an edition to the ensemble. The GEO algorithm was implemented in a modular manner which can easily use any model as ensemble constituents. Since ensembles thrive when using a diversity of algorithms, adding

neural networks or other algorithms to the mix would likely increase the accuracy across train, cross-validation and test sets.

Testing different blending models or even an ensemble of blending models could also bear fruit. In the current implementation, a uniform vote is used to classify the ensemble's predictions. Blending models can easily be swapped out in this implementation. Other simple blending models could be tested such as taking the average of each model's output as the ensemble prediction or feeding predictions through a sigmoid filter then taking the simple average of their output. Optimization models such as neural networks or logistic regression models could also be fit as blending models, at a cost of using more computation time to fit the models. The GEO's results generalizing across multiple blending algorithms would further affirm its outperformance.

More datasets could be used to test this implementation of the Generalized Ensemble Optimization. The GEO was coded in a way in which the dataset used to optimize and predict upon could be exchanged for any other one. If the GEO outperforms more datasets, its viability will be further confirmed.

The GEO could also be tested for different use cases than machine learning optimization. Since the models and datasets could easily be swapped out, the GEO could be used to build portfolios (ensembles) of systematic trading strategies. Systematic trading strategies, like machine learning models have hyperparameters which each effect the systems' equity lines. Machine learning models' out of sample predictions and systematic trading strategies' future performance are also similar: having multiple uncorrelated predictors (alphas) can substantially boost model performance. Instead of maximizing cross-validation binary accuracy as in this

implementation, the GEO could maximize the ensemble portfolio's cross-validation returns, Sharpe Ratio or Omega Ratio.

References

AlphaGo. (2017, October 18). Retrieved from https://deepmind.com/research/alphago/

Applying Genetic Algorithms to Define a Trading System. (2018, January 15). Retrieved from https://quantdare.com/ga-to-define-a-trading-system/

Bhande, A. (2018, March 11). What Is Underfitting And Overfitting In Machine Learning and How To Deal With It. Retrieved from https://medium.com/greyatom/what-is-underfitting-and-overfitting-in-machine-learning-and-how-to-deal-with-it-6803a989c76

Chen, L. (2019, January 7). Support Vector Machine - Simply Explained. Retrieved from https://towardsdatascience.com/support-vector-machine-simply-explained-fee28eba5496

Chen, S., & Navet, N. (2007). Failure of Genetic-Programming Induced Trading Strategies: Distinguishing between Efficient Markets and Inefficient Algorithms. Computational Intelligence in Economics and Finance. doi:10.1007/978-3-540-72821-4_11

Chen, T. (2014, October 22). Introduction to Boosted Trees. Retrieved from https://homes.cs.washington.edu/~tqchen/pdf/BoostedTree.pdf

Craib, R. (2018, March 22). Encrypted Data For Efficient Markets. Retrieved from https://medium.com/numerai/encrypted-data-for-efficient-markets-fffbe9743ba8

Craib, R. (2016, August 30). Super Intelligence for The Stock Market. Retrieved from https://medium.com/numerai/invisible-super-intelligence-for-the-stock-market-3c64b57b244c

C-Support Vector Classification. (n.d.). Retrieved from https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html

Dalio, R. (2017). Principals. Simon & Schuster.

Google Developers (2018, October 12). Logistic Regression Retrieved from https://developers.google.com/machine-learning/crash-course/logistic-regression/video-lecture

Hinton, G., Srivastava, N., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2013). Dropout: A Simple Way to Prevent Neural Networks from Overfitting.

How Numerai Works. (2019). Retrieved from https://numer.ai/learn

Jaderberg, M., Dalibard, V., Osindero, S., Czarnecki, W., Donahue, J., Razavi, A., . . . Kavukcuoglu, K. (2017). Population Based Training of Neural Networks. DeepMind, London, UK.

Kraus, M. (2019, March 28). Using Bayesian Optimization to reduce the time spent on hyperparameter tuning. Retrieved from https://medium.com/vantageai/bringing-back-the-time-spent-on-hyperparameter-tuning-with-bayesian-optimisation-2e21a3198afb

Morde, V. (2019, April 8). XGBoost Algorithm: Long May She Reign! Retrieved from https://towardsdatascience.com/https-medium-com-vishalmorde-xgboost-algorithm-long-she-may-rein-edd9f99be63d

Nagpal, A. (2017, October 14). L1 and L2 Regularization Methods. Retrieved from https://towardsdatascience.com/l1-and-l2-regularization-methods-ce25e7fc831c

Overing, M. (2019). Dissertation Proposal.

Panchekha, A., Tull, R. S., Jr., & Bell, M. (2018). Ensemble Active Management the Next Evolution in Investment Management. EAM Research Consortium.

Patel, S. (2017, May 4). SVM (Support Vector Machine) - Theory. Retrieved from https://medium.com/machine-learning-101/chapter-2-svm-support-vector-machine-theory-f0812effc72

Paul, S. (2018, August 18). Hyperparameter Optimization in Machine Learning. Retrieved from https://www.datacamp.com/community/tutorials/parameter-optimization-machine-learning-models

Sylvester, J., & Chawla, N. V. (2005). Evolutionary Ensembles: Combining Learning Agents Using Genetic Algorithms. Department of Computer Science and Engineering, University of Notre Dame.

XGBoost Parameters¶. Retrieved from https://xgboost.readthedocs.io/en/latest/parameter.html

Zulkifli, H. (2018, January 27). Understanding Learning Rates and How It Improves Performance in Deep Learning. Retrieved from https://towardsdatascience.com/understanding-learning-rates-and-how-it-improves-performance-in-deep-learning-d0d4059c1c10